

Progetto JPokeBattle

Francesco Giammaria

matricola: 2108040

Specifiche Selezionate

Ho realizzato il progetto in **sviluppatore singolo**. Ho implementato tutte le Features Minime e Tipiche mentre di quelle Extra ho selezionato:

- ✓ **Set Crescita** (punti allenamento, punti individuali, salti di livello evoluzioni e apprendimento mosse)
- ✓ **Set TUI** (implementare una modalità di gioco testuale via terminale, selezionabile in alternativa alla GUI con un'opzione al lancio dell'applicazione)

Ho anche realizzato, fuori dalle Features sviluppatore singolo, la Two-player mode per permettere lotte tra due giocatori umani.

Design Patterns Utilizzati

Pattern MVC: Ho utilizzato il pattern Model-View-Controller per la realizzazione della schermata della Battaglia.

-model: nel package project.gui sono presenti diversi file con suffisso Model, questo perché volevo accomunare il più possibile le due diverse modalità di gioco (giocatore singolo o due giocatori) senza però portare a ridondanza del codice. Dunque quello che ho fatto è stato creare una superclasse astratta **BattagliaBaseModel** che contiene tutti i metodi comuni ad entrambe le modalità (getter, setter, inizia, eseguiMossa, tentaFuga etc...), e che sarà poi estesa dalle sottoclassi concrete **BattagliaModel** che gestisce la modalità giocatore singolo 1 vs Bot, e **Battaglia1v1Model** che gestisce invece le lotte 1v1 tra due giocatori umani.

Questo approccio garantisce riutilizzo e flessibilità del codice (se in futuro volessi aggiungere una nuova modalità basterebbe creare una nuova sottoclasse).

-view: implementata nella classe **BattagliaView** essa è la schermata che verrà visualizzata dall'utente e che potrà essere utilizzata sia per le lotte classiche che per quelle a due giocatori. È stato necessario solo un piccolo adattamento aggiungendo un metodo per mostrare o meno i pulsanti di g2 a seconda della modalità di gioco in cui ci troviamo. Tuttavia questo metodo non viene MAI chiamato dalla view stessa, ma solo dal Controller per garantire la divisione delle responsabilità del pattern MVC.

Fondamentale specificare che per la realizzazione di alcune schermate più complesse (come appunto la BattagliaView) è stato utilizzato il plug-in di **eclipse WindowBuilder**

-controller: gestisce le interazioni tra il model e la view. In particolare **BattagliaController** riceve come model un oggetto di tipo BattagliaBaseModel, consentendo le interazioni con la view sia per la modalità single player che per quella a due giocatori.

Pattern Observer: fondamentale per il funzionamento del pattern MVC. Lo ho usato per notificare gli aggiornamenti del model alla view.

-Observer: la BattagliaView implementa l'interfaccia Observer e sovrascrive il metodo update così da assicurare che tutte le componenti (barre della vita, sprites, nomi, livelli etc) vengano aggiornati adeguatamente.

-Observable: la classe BattagliaBaseModel estende Observable (di conseguenza viene ereditato anche dalle sottoclassi BattagliaModel e Battaglia1v1Model) e notifica i cambiamenti agli osservatori (cioè la view) tramite il metodo setStato(String messaggio) che oltre a richiamare il metodo update della view permette di stampare sul logBattaglia un messaggio specificato come parametro.

Pattern Singleton: Lo ho utilizzato nella classe **Pokedex** così da garantire un'unica istanza "globale" condivisa da tutte le classi. Fondamentale per garantire la corretta creazione dei pokemon tramite il secondo costruttore (vedremo più avanti).

Salvataggio dei Dati

Per il salvataggio dei dati ho deciso di utilizzare l'interfaccia `Serializable`. **La classe principale che verrà salvata è `Giocatore` che**, oltre al nome del giocatore, **porta con sé anche la squadra e la collezione** (e quindi tutti quanti i pokemon sbloccati fino a quel momento, con i relativi livelli, mosse apprese, evoluzioni e statistiche).

Per dividere al meglio le responsabilità ho creato anche una classe di servizio **`GestoreSalvataggio`** che fornisce metodi statici per salvare, caricare e cancellare i dati relativi al giocatore (tramite serializzazione e deserializzazione come detto prima).

Tuttavia, affinché `Serializable` funzioni, è necessario che tutte le classi coinvolte implementino l'interfaccia e che la classe dell'oggetto salvato abbia un costruttore vuoto per permettere la deserializzazione. Pertanto **`Giocatore` ha due costruttori**: il primo vuoto per i motivi detti prima ed il secondo che invece accetta parametri e richiama il costruttore di `Allenatore` tramite `super`.

GestoreFonts e MyPanel

`GestoreFonts` è una classe che ho realizzato per caricare dei font personalizzati utilizzati nelle label della GUI. Essi sono accessibili tramite i metodi statici `getPokemonSolidFont` e `getPokemonHollowFont`.

`MyPanel`, altrettanto, è una classe astratta che ho utilizzato per realizzare la GUI. Questa classe estende `JPanel` permettendo di impostare un'immagine di sfondo semplicemente fornendo il path dello sfondo al costruttore. Inoltre aggiunge un metodo `switchPanel` che consente di passare da un panel all'altro semplicemente fornendo il nome di quello a cui si vuole "andare".

Questo torna utile perché l'intero gioco è visualizzato tramite un unico frame che passa da un panel all'altro utilizzando un `CardPanelLayout`. Ciascuno dei Panel contenuti all'interno del `cardPanel` estende `MyPanel` così da ereditare il metodo per passare ai prossimi.

public abstract class Allenatore implements Serializable

Questa classe astratta contiene una serie di metodi comuni alle due sottoclassi concrete `Giocatore` e `Bot` con l'intento di ridurre il più possibile la **ridondanza del codice**.

Inoltre, quest'implementazione torna utile anche per la realizzazione delle lotte 1v1 a due giocatori. Infatti all'interno della classe `BattagliaBase` il **`giocatore2`** (cioè l'avversario) viene specificato di tipo `Allenatore`, perché potrà essere sia un `Giocatore` che un `Bot` a seconda della modalità di gioco concreta selezionata.

Ciascun **`Allenatore`** avrà a sé associati vari campi quali:

- **`String nome`** -> *(banalmente il nome dell'allenatore che verrà visualizzato in battaglia)*
- **`Pokemon pokemonAttuale`** -> *(il pokemon attualmente in campo).*
- **`List<Pokemon> squadra`** -> *(la lista di 6 pokemon selezionati dall'allenatore per combattere)*
- **`List<Pokemon> collezione`** -> *(la lista di pokemon sbloccati che possono essere aggiunti alla squadra. Ho scelto una List piuttosto che un set perché voglio consentire le ripetizioni. È possibile aver catturato 2 charmander e 3 bulbasaur)*

public class Pokemon implements Serializable

Questa è ovviamente la classe principale e più grande di tutto il progetto, soprattutto perché ho deciso di implementare il **Set Crescita** (campo Extra delle Feature Comuni) che mi ha portato ad inserire molti campi per la gestione dei punti individuali (IV) e punti allenamento (EV). Dunque, per non avere un unico costruttore con tantissimi parametri, ho deciso di aggiungere un secondo costruttore, che fa uso della Classe Pokedex per creare le istanze.

- **Primo Costruttore:** prende tutti i parametri (nome, tipo1, tipo2, mosse, mosseDaImparare, lv, ivAttacco, ivDifesa, evAttacco, evDifesa, etc...) e viene utilizzato per riempire il pokedex.
- **Secondo Costruttore:** prende come unico parametro il nome del pokemon e viene utilizzato generalmente per creare le istanze. Questo secondo costruttore cerca il nome del pokemon nel pokedex (se non lo trova viene lanciata un'eccezione) e restituisce l'istanza "clone" di quella trovata.

Credo che questo approccio renda la creazione dei pokemon, delle squadre e dei livelli molto più semplice e chiara semplicemente scrivendo **Pokemon charmander = new Pokemon("Charmander")** anziché fornendo ogni volta tutte le statistiche.

Set Crescita

Per i salti di livello la classe è dotata di alcuni metodi specifici per il calcolo delle statistiche tramite:

- Punti Individuali - IV (che vengono stabiliti causalmente alla creazione del pokemon)
- Punti Allenamento - EV (che sono inizialmente tutti a 0 ed aumentano man man che si vincono le battaglie in base alla difficoltà dei nemici sconfitti).

Analogamente a quanto avviene nei giochi Pokemon ufficiali.

Per le evoluzioni ciascun pokemon ha un `HashMap<int, String>` associata chiamata "evoluzioni" che ha come chiavi il lv a cui il pokemon si evolve e come valore il nome dell'evoluzione che verrà poi sostituito all'attuale nome del pokemon.

Per le mosse ho adottato sostanzialmente lo stesso approccio della mappatura dei livelli. Nel caso in cui il pokemon abbia già 4 mosse disponibili viene chiamato un ulteriore metodo che permette all'utente di decidere quale mossa dimenticare per apprendere quella nuova.

Di questi metodi `levelUp()`, `evolvi()` e `imparaMossa()` ho implementato anche la **versione AUTO**, cioè quella automatica che non prevede alcun'interazione con l'utente e serve esclusivamente per la creazione delle squadre dei bot.

public class Pokedex

Per realizzarla ho deciso di utilizzare il **Design Pattern Singleton** creando una sola istanza globale di Pokedex (che è nella pratica un `HashMap<String, Pokemon>`) unica e condivisa tra tutte le classi. Come anticipato prima questa classe funziona come una sorta di database che contiene un'istanza "base" per ogni pokemon disponibile nel gioco.

In questo modo, quando andremo a creare un nuovo pokemon tramite il nome passato come parametro al secondo costruttore, andremo a prelevare l'istanza "base" dal Pokedex e ne copieremo tutti gli attributi per restituire la nuova istanza.

```
public Pokemon(String nome) {
    Pokedex pokedex = Pokedex.getInstance();
    Pokemon p = pokedex.get(nome);
    if (p != null) {
        this.nome = p.nome;
        this.tipo1 = p.tipo1;
        this.tipo2 = p.tipo2;
        this.mosse = new ArrayList<>(p.mosse);
        this.mosseDaImparare = new HashMap<>(p.mosseDaImparare);
        this.evoluzioni = new HashMap<>(p.evoluzioni);
        etc... etc... etc...
```

public enum *Elementi*

Ad ogni elemento viene associata una string “descrizione” che serve per la rappresentazione tramite toString().

Per quanto riguarda le meccaniche tra i tipi (nel senso di debolezze, resistenze e immunità) ho deciso di inserirle all'interno di 3 diverse **private static final HashMap<Elementi, HashSet<Elementi>>** (dal momento che sono comuni a tutte le istanze e non voglio vengano modificate in futuro). Per cui:

- Nella HashMap **resistenze** avrò: **TERRA, (VELENO, ROCCIA)**
- Nella HashMap **debolezze** avrò: **TERRA, (ACQUA, ERBA, GHIACCIO)**
- Nella HashMap **immunità** avrò: **TERRA, (ELETTRO)**

public class Livello

La classe Livello **mantiene tutte le informazioni relative all'avversario Bot** che si andrà a scontrare nelle Battaglie Classiche ad 1 giocatore. Ogni livello viene costruito a partire dal nome, in base al quale verrà creato il Bot e la relativa squadra da affrontare.

Per quanto riguarda la modalità di gioco TUI viene fornito un metodo statico getNomeLivello(int i) che consente di prendere il nome e creare il livello tramite semplicemente l'indice. In questo modo sarà più facile per l'utente decidere di affrontare il lv 1, il lv 2 etc... piuttosto che inserire manualmente il nome del livello (cosa che può facilmente portare ad errori anche solo sbagliando una lettera).

public abstract class BattagliaBase

Le classi qui sotto sono utilizzate SOLO SE all'avvio del gioco si decida di giocare in **modalità TUI**.
Altrimenti si fa riferimento alle Battaglie Model e al pattern MVC.

BattagliaBase è una classe astratta che fornisce una serie di metodi comuni a tutti i tipi di battaglia “concrete”. È lo stesso concetto dei Model, ho una superclasse astratta con metodi comuni che verrà poi estesa dalle singole modalità di gioco che decido di implementare.

Questa “tecnica” permette anche una certa flessibilità e riutilizzo del codice: se in futuro volessi aggiungere altre modalità basterà creare una sottoclasse di BattagliaBase che eredita tutti i metodi e implementa le sue meccaniche specifiche. Al momento però mi sono limitato alle sole modalità:

- **Battaglia:** gestisce la battaglia classica tra un giocatore umano e un bot.
- **Battaglia1v1:** gestisce la battaglia tra due giocatori umani.

public class JPokeBattle

All'interno del package project.main è contenuta la classe JPokeBattle che serve sostanzialmente ad avviare il programma, inizializzando il frame principale e caricando i dati del giocatore.